

Project Demo Notes

eBPF security monitoring tool

Politecnico di Torino

Rakuten Mobile

Simone Romano Giuseppe Insalaco

May 2026

Introduction

This document is meant as a lightweight support for the first technical demo. Rather than presenting the whole project from scratch, it focuses on the current state of the prototype: what is already working, how the runtime path is organized, and which parts are useful to observe during the live execution.

1 Goal

This demo shows the current end-to-end pipeline:

- selected kernel hooks are attached from Go;
- events are sent through ring buffer or perf buffer;
- userspace decodes raw bytes into structured events;
- filters reduce noise;
- output is printed as `table` or `json`.

This is not yet a detection engine. The goal of the demo is to show that the event collection pipeline works and that the architecture is ready for enrichment and detection logic.

2 Current Pipeline

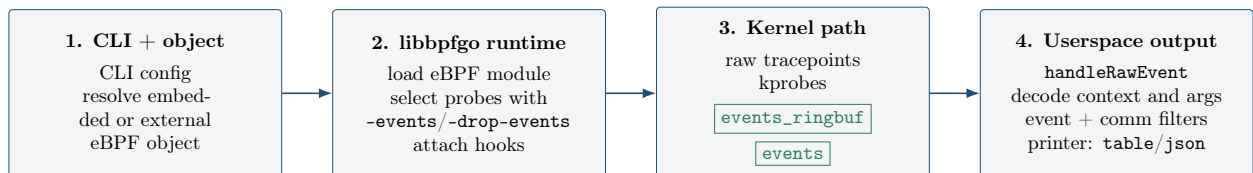


Figure 1: Current execution pipeline of the tool.

3 What Works Today

- eBPF object build through `make bpf`.
- Go userspace runtime based on `libbpfgo`.
- Probe registry in `pkg/ebpf/probes`.
- Event selection with `-events` and `-drop-events`.

- Command-name filtering with `-comms`.
- Ring buffer reader for process/security events.
- Perf buffer reader for Tracee-like/networking events.
- Output layer with `json` and `table`.

4 Event Groups

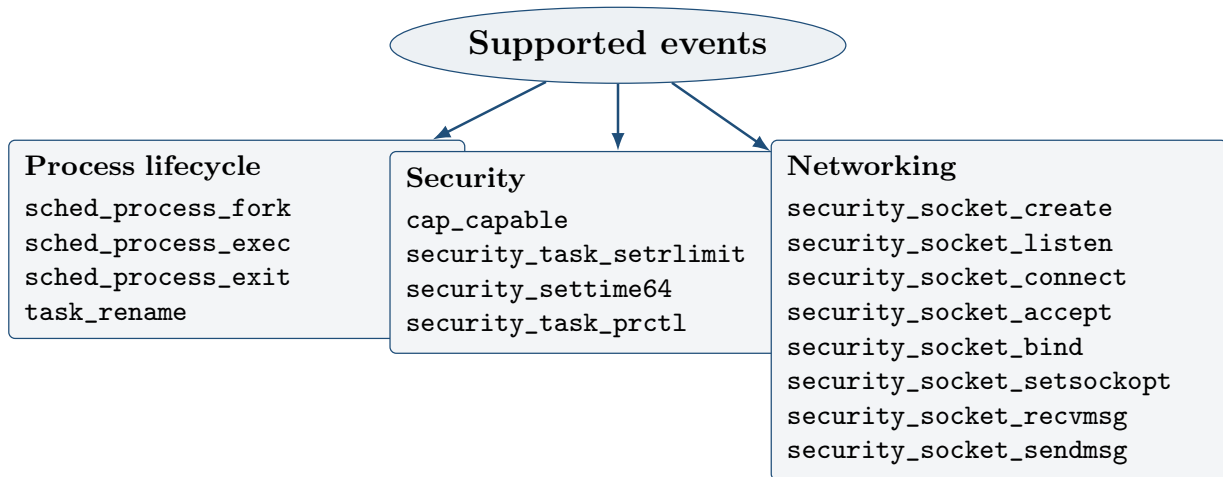


Figure 2: Supported event families.

5 Demo Flow

5.1 1. Build the Tool

```
cd demo_project
make build
```

Expected result:

```
dist/project.bpf.o
dist/project
```

5.2 2. Show Process Execution Events

Terminal 1:

```
sudo ./dist/project \
  --events task_rename,sched_process_exec,sched_process_exit \
  --comms ls,whoami \
  --output table
```

Terminal 2:

```
ls
whoami
```

Expected output shape:

```
event=task_rename pid=... tid=... uid=1000 comm=bash args=old_name=bash,new_name=ls
event=sched_process_exec pid=... tid=... uid=1000 comm=ls args=filename=/usr/bin/ls
event=sched_process_exit pid=... tid=... uid=1000 comm=ls args=exit_code=0,group_dead=1
```

What to explain:

- `task_rename` shows the task `comm` transition.
- `sched_process_exec` shows the executed binary path.
- `sched_process_exit` shows process termination and exit code.
- `-comms` keeps the output focused on the commands used in the demo.

5.3 3. Show Networking Event Path

Terminal 1:

```
make filtered
```

Terminal 2:

```
python3 -m http.server 18080 --bind 127.0.0.1
```

Terminal 3:

```
curl http://127.0.0.1:18080
```

What to explain:

- `security_socket_connect` is attached as a kprobe.
- Networking hooks currently follow the perf-buffer path.
- The runtime now reads both ring buffer and perf buffer.

5.4 4. Optional: Show a Sensitive Security Hook

Terminal 1:

```
sudo ./dist/project --events security_settime64 --output table
```

Terminal 2:

```
sudo date -s "$(date '+%Y-%m-%d %H:%M:%S')"
```

What to explain:

- `security_settime64` is triggered when a process tries to modify system time.
- It requires privileges such as `CAP_SYS_TIME`.
- Use this only on a test VM.

6 Architecture Talking Points

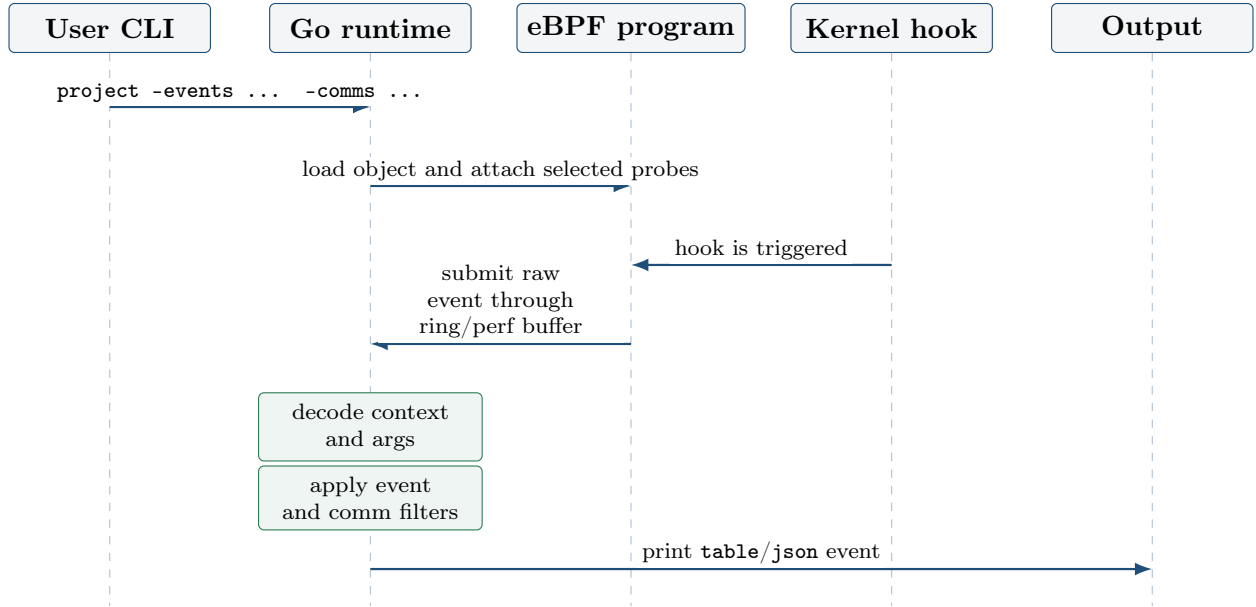


Figure 3: High-level interaction between CLI, Go runtime, eBPF code, kernel hooks and output.

Key points:

- The tool does not attach every probe blindly when `-events` is used.
- The decoder is intentionally separated from output formatting.
- The output layer can evolve without changing the eBPF runtime.
- The current design is close to Tracee conceptually, but smaller and easier to reason about for the thesis.

7 Known Limits

- Some arguments are still raw numeric values.
- `security_task_prctl` needs symbolic mapping for options like `PR_SET_VMA`.
- Socket addresses and socket constants need better human-readable formatting.
- `-comms` is a userspace filter, so it improves output readability but does not reduce kernel-side work.
- The final transport strategy still needs consolidation: keep both ring/perf buffers, or converge to perf buffer in a more Tracee-like design.
- Detection logic and MITRE ATT&CK mapping are future work.

8 Short Closing

The current version demonstrates the hard part of the project: a working kernel-to-userspace event pipeline on the target Rocky Linux kernel. The next step is not loading eBPF anymore, but improving event semantics: better argument mapping, stronger filters and eventually detection logic.